NTNU

Physics-Informed Neural Networks

One method for the course benchmarks: heat, surface vessels, wind

Adil Rasheed · Part V — Hybrid models
NTNU — Department of Engineering Cybernetics

Outline

- Motivation
- Problem Setting: ODEs & PDEs
- Automatic Differentiation
- The Core PINN Algorithm
- One Recipe, Three Benchmarks
- Heat Diffusion (PDE)
- Autonomous Surface Vessels (ODEs)
- Wind Turbine and Wake (ODE + PDE)
- Failure Modes & Remedies
- Algorithmic Variants
- Practical Guidelines & Outlook

01

Motivation

badly; Module 20 kept the solver and learned a correction; Module 21 recovered equations from data. This module puts a *known* equation into the loss of a network.

Most engineering models live at one of two unsatisfactory extremes:

Physics-based simulation

- Governing equations, first principles
- Predictive, interpretable, extrapolates
- Expensive: hours–weeks per run
- Struggles with unmodeled effects and unknown parameters

Pure data-driven ML

- Fast inference once trained
- Captures what the equations miss
- Data-hungry; poor extrapolation
- No conservation laws, no causality, fails silently

Physics-informed neural networks (PINNs) put the governing equations *inside the loss function* — physics acts as a regularizer, so far fewer, noisier data can suffice.

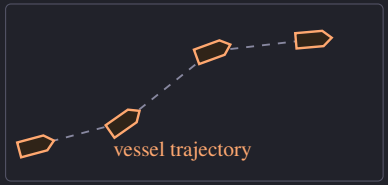
One method, two faces

The same loss function solves problems in completely different domains:



Wind engineering

fields $\boldsymbol{u}(\boldsymbol{x}, t)$ — *PDEs*



Autonomous surface vessels

trajectories $\boldsymbol{u}(t)$ — *ODEs*

$$\mathscr{L}(\boldsymbol{\theta}) = \mathscr{L}_{\text{physics}} + \mathscr{L}_{\text{initial}} + \mathscr{L}_{\text{boundary}} + \mathscr{L}_{\text{data}}$$ (boundary term: PDEs only)

Lecture roadmap

Part I — The method
application-agnostic

1. Motivation
2. Problem setting & the loss
3. Automatic differentiation
4. The core algorithm

Part II — The benchmarks
simple → realistic

5. One recipe, three benchmarks
6. Heat diffusion — *PDE*
7. Surface vessel — *ODEs*
8. Wind turbine & wake

Part III — Beyond vanilla
what breaks, what fixes it

9. Failure modes & remedies
10. Variants: cPINN, XPINN, vPINN, ensembles
11. Practical guidelines

How to read this lecture

Part I builds the method once. Part II applies the identical recipe to three benchmarks, from a linear PDE with an exact solution to real-world data. Part III explains the failures you will have seen by then, and the variants that fix them.

02

Problem Setting: ODEs & PDEs

Notation used throughout

Typography conventions

Scalars italic lowercase (u, x, t); vectors bold italic lowercase ($\boldsymbol{u}, \boldsymbol{x}, \boldsymbol{w}, \boldsymbol{\nu}, \boldsymbol{\eta}$); matrices bold upright uppercase ($\mathbf{M}, \mathbf{R}, \mathbf{D}$); sets calligraphic ($\mathscr{D}, \mathscr{L}$); operators roman ($\mathrm{N}, \mathrm{B}$).

Symbol roles — one meaning each

- t: time | $\boldsymbol{x}$: spatial coordinate (PDEs) / position | $\boldsymbol{\lambda}$: physical parameters
- $\boldsymbol{u}$: the *unknown* the network computes — state $\boldsymbol{u}(t)$ (ODE) or field $\boldsymbol{u}(\boldsymbol{x}, t)$ (PDE); output $\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}$
- $\boldsymbol{w}$: known input / control (ODEs only)

Color semantics

Blue physics | Orange boundary | Violet initial | Green data | Amber warnings | Red failures

Dynamical systems: ordinary differential equations

The unknown is a *trajectory* $\boldsymbol{u}(t) \in \mathbb{R}^n$ over a time interval $[0, T]$:

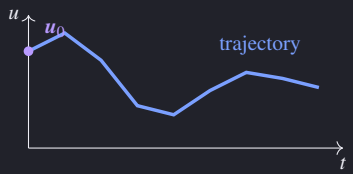
ODE initial-value problem

$$\frac{d\boldsymbol{u}}{dt} = \boldsymbol{f}\big(\boldsymbol{u}(t), \boldsymbol{w}(t); \boldsymbol{\lambda}\big), \qquad \boldsymbol{u}(0) = \boldsymbol{u}_0$$

$\boldsymbol{f}$: system dynamics; $\boldsymbol{w}(t)$: known input / control; $\boldsymbol{\lambda}$: physical parameters.

Examples

- Pendulum: $\ddot{\phi} + c\dot{\phi} + \frac{g}{\ell}\sin\phi = 0$
- Mass–spring–damper, robot arms
- Marine vessel maneuvering (Part II)



Initial-value problem: no boundary conditions — only an IC; the "domain" is just $[0, T]$. (ODE boundary-value problems do have them.)

Field problems: partial differential equations

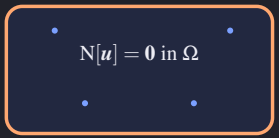
The unknown is a *field* $\boldsymbol{u}(\boldsymbol{x},t)$ over a spatio-temporal domain $\mathscr{D} = \Omega \times [0,T]$:

PDE problem

$$\mathrm{N}\left[\boldsymbol{u}(\boldsymbol{x},t);\boldsymbol{\lambda}\right] = \boldsymbol{0} \ \text{ in } \mathscr{D}, \qquad \mathrm{B}\left[\boldsymbol{u}\right] = \boldsymbol{0} \ \text{ on } \partial\Omega, \qquad \boldsymbol{u}(\boldsymbol{x},0) = \boldsymbol{u}_0(\boldsymbol{x})$$

Examples

- Heat / diffusion: $\partial_t u = \alpha \nabla^2 u$
- Advection–diffusion, wave equation
- Navier–Stokes, RANS, ABL flows (Part II)



Now boundary conditions join initial conditions — the field must be pinned on $\partial\Omega$ for the problem to be well-posed.

One residual to rule them all

Both problem classes reduce to: *find the function that makes a residual vanish*.

	ODE	PDE
Unknown	trajectory $\boldsymbol{u}(t)$	field $\boldsymbol{u}(\boldsymbol{x},t)$
Domain	interval $[0,T]$	$\Omega \times [0,T]$
Residual	$\dot{\boldsymbol{u}} - \boldsymbol{f}(\boldsymbol{u},\boldsymbol{w};\boldsymbol{\lambda})$	$\mathrm{N}[\boldsymbol{u};\boldsymbol{\lambda}]$
Initial condition	$\boldsymbol{u}(0) = \boldsymbol{u}_0$	$\boldsymbol{u}(\boldsymbol{x},0) = \boldsymbol{u}_0(\boldsymbol{x})$
Boundary condition	none (initial-value problem)	$\mathrm{B}[\boldsymbol{u}] = \boldsymbol{0}$ on $\partial\Omega$
Derivative orders	usually 1st–2nd in t	mixed, often 2nd in $\boldsymbol{x}$

The PINN idea in one sentence

Replace the unknown function by a neural network $\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}$, evaluate the residual on it via automatic differentiation, and minimize the residual norm over the network parameters $\boldsymbol{\theta}$.

Anatomy of the PINN loss

Write every condition as a *residual*: move all terms to the left-hand side, so that each residual is zero exactly when its condition holds.

PDE $r(\boldsymbol{x},t) = \mathrm{N}\big[\hat{\boldsymbol{u}}_{\boldsymbol{\theta}};\boldsymbol{\lambda}\big](\boldsymbol{x},t)$ in $\Omega \times (0,T]$

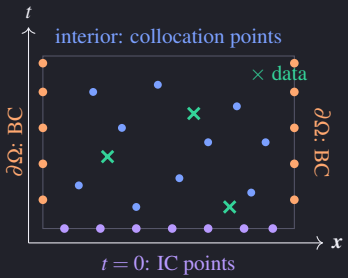
IC $r_i(\boldsymbol{x}) = \hat{\boldsymbol{u}}_{\boldsymbol{\theta}}(\boldsymbol{x},0) - \boldsymbol{u}_0(\boldsymbol{x})$ in Ω

BC $r_b(\boldsymbol{x},t) = \mathrm{B}\big[\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}\big](\boldsymbol{x},t)$ on $\partial\Omega$

Data $r_{d,k} = \hat{\boldsymbol{u}}_{\boldsymbol{\theta}}(\boldsymbol{x}_k,t_k) - \boldsymbol{u}_k^{\text{data}}$ at sensors

Each residual is squared and averaged over its own point set:

$$\mathscr{L}(\boldsymbol{\theta}) = \frac{w_r}{N_r}\sum_j \|r\|^2 + \frac{w_i}{N_i}\sum_j \|r_i\|^2 + \frac{w_b}{N_b}\sum_j \|r_b\|^2 + \frac{w_d}{N_d}\sum_k \|r_{d,k}\|^2$$



Forward problems need no green term; initial-value ODEs have no orange one. Collocation points need no labels — only computation.

Forward vs. inverse problems

Forward

Physics fully specified $\rightarrow$ *find the solution*.

$$\min_{\boldsymbol{\theta}} \mathcal{L}_r + \mathcal{L}_i [+ \mathcal{L}_b]$$

- Needs *no* measured data
- Output: fast, differentiable surrogate

Inverse

Sparse data, unknown parameters $\rightarrow$ *find both*.

$$\min_{\boldsymbol{\theta},\boldsymbol{\lambda}} \mathcal{L}_r(\boldsymbol{\theta},\boldsymbol{\lambda}) + \mathcal{L}_d(\boldsymbol{\theta})$$

- $\boldsymbol{\lambda}$ trained *alongside* $\boldsymbol{\theta}$
- Output: solution + identified physics

Part II preview: one unknown $\boldsymbol{\lambda}$ per benchmark

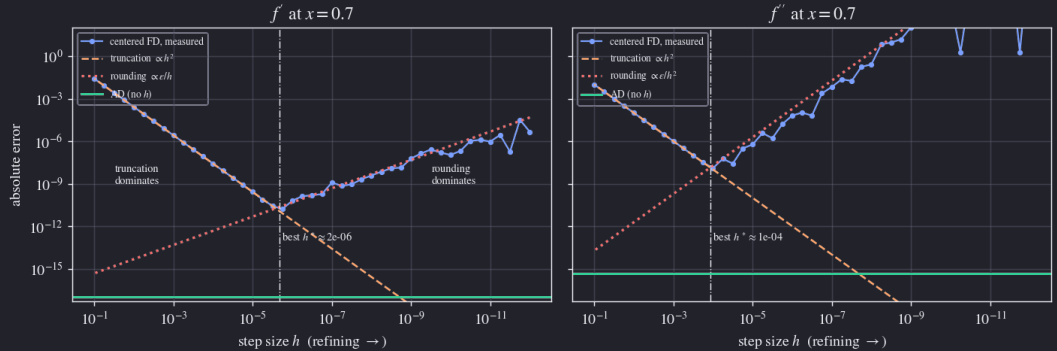
Heat: diffusivity α. Vessel: damping d_u, drag c. Wind: drivetrain inertia J, eddy viscosity ν_t.

03

Automatic Differentiation

Three ways to differentiate

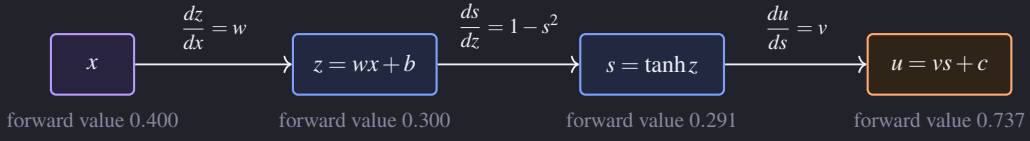
Analytic: exact, needs a formula. **Finite differences (FD)**: need a step h. **AD**: the chain rule on the code, no h.



FD for f': $E(h) \approx \frac{h^2}{6}|f'''| + \frac{\varepsilon}{h}|f|$. Truncation is a power law: a straight line of slope 2 on log–log axes. Rounding grows as $h \to 0$: $f(x+h) - f(x-h)$ subtracts nearly equal numbers, then divides by h. Refining helps only down to $h^* \approx 2 \times 10^{-6}$ (f') and 10^{-4} (f''). AD: 4×10^{-16}.

AD is the chain rule on a computation graph

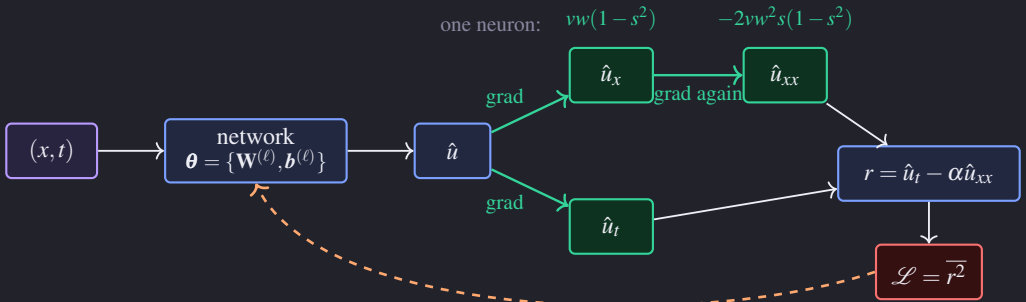
One neuron, $u = v\tanh(wx+b)+c$, with $v = 1.5$, $w = 2$, $b = -0.5$, $c = 0.3$, evaluated at $x = 0.4$. Its hidden value is called $s = \tanh z$ (not the step h of the previous slide).



$$\frac{du}{dx} = v \cdot (1 - s^2) \cdot w = 1.5 \times 0.915 \times 2 = 2.745, \qquad \frac{d^2u}{dx^2} = \frac{d}{dx}\left[vw(1-s^2)\right] = -2\,v\,w^2\,s\,(1-s^2) = -3.199$$

Hand calculation and `torch.autograd.grad` agree to all 12 printed digits.

Two derivative directions, one engine



Green: one value per collocation point; each `grad` uses `create_graph=True`, so $\hat{u}_x$ can be differentiated again into $\hat{u}_{xx}$.

Orange: one value per weight and bias; it must pass *through* the green nodes. Measured: the output bias gets *no* gradient from $\overline{r^2}$ — only IC/BC terms fix the solution's level.

The residual in PyTorch, annotated

```
x = torch.rand(N, 1, requires_grad=True)     # record ops on the inputs
t = torch.rand(N, 1, requires_grad=True)
u = net(torch.cat([x, t], 1))                 # shape (N, 1)
ones = torch.ones_like(u)                     # vector-Jacobian weights
u_t,  = torch.autograd.grad(u,   t, ones, create_graph=True)
u_x,  = torch.autograd.grad(u,   x, ones, create_graph=True)
u_xx, = torch.autograd.grad(u_x, x, ones, create_graph=True)
loss = (u_t - alpha * u_xx).pow(2).mean()
loss.backward()                               # dL/dtheta, through u_xx
```

`grad_outputs = ones`

Returns $\partial(\sum_i u_i)/\partial x_j$. That equals the per-sample $\partial u_j/\partial x_j$ only if samples do not interact (no batch norm). For a vector output it *sums* the components.

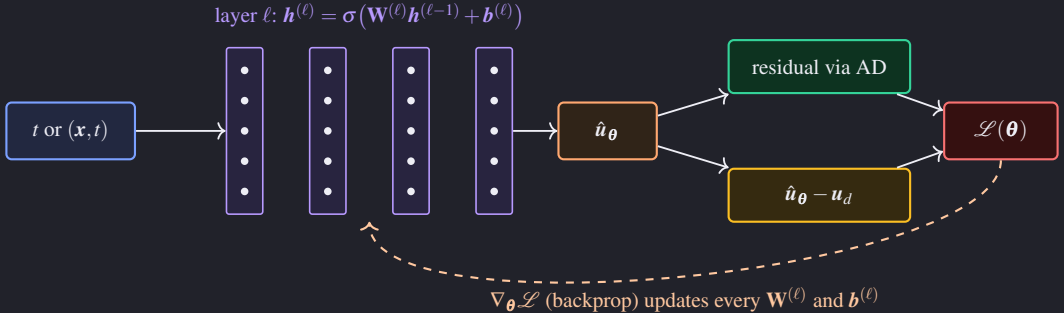
`create_graph` vs `retain_graph`

`create_graph=True` makes the derivative differentiable. `retain_graph=True` only keeps the old graph for a second backward pass — $\hat{u}_{xx}$ would then fail.

04

The Core PINN Algorithm

PINN architecture at a glance



- Parameters: $\boldsymbol{\theta} = \{\mathbf{W}^{(\ell)}, \boldsymbol{b}^{(\ell)}\}_{\ell=1}^{L}$, all weight matrices and bias vectors.
- Forward pass: inputs $\rightarrow \hat{\boldsymbol{u}}_{\boldsymbol{\theta}}$ — a plain MLP, nothing exotic.
- AD pass: build the ODE/PDE residual at collocation points.
- Backward pass: $\nabla_{\boldsymbol{\theta}}\mathscr{L}$ updates the weights.

Where to enforce the physics: collocation points

The residual is enforced at sampled points — the strategy matters.

Uniform / grid



Simple; wasteful in smooth regions.

Latin hypercube



Low-discrepancy coverage; good default.

Adaptive (RAR)



Concentrate where the residual is largest.

Residual-based adaptive refinement (RAR)

Periodically evaluate the residual on a dense candidate pool; add the top-k highest-residual points to the training set. Essential for shocks, fronts, boundary layers — and for stiff transients in ODEs.

Optimization: a two-stage recipe

1. **Adam (first-order)**
   - Robust to noisy gradients; escapes poor initializations
   - Typical: 10^4–10^5 steps, learning rate 10^{-3} with decay
2. **L-BFGS (quasi-Newton)** for refinement
   - Exploits curvature; needs a *fixed* collocation set (deterministic loss)
   - Sensitive to initialization $\rightarrow$ run *after* Adam
   - Measured (heat, 1-D): 320 L-BFGS evaluations cut the relative error $4.7 \times 10^{-3} \rightarrow 3.1 \times 10^{-4}$; the same budget of continued Adam gives 2.1×10^{-3}

Activation functions: smooth to the residual's order

A strong-form residual differentiates the network — activations must be smooth to that order. Use tanh, sin, Swish/GELU. **No ReLU** for second-order residuals: $\sigma'' \equiv 0$, so the term silently collapses (Section 3).

- Initialize with Glorot/Xavier; non-dimensionalize inputs and outputs first.

Pseudocode: the training loop

Algorithm 1: PINN training (ODE or PDE)

Input: residual operator, IC (and BCs if PDE), optional data, weights $w_\bullet$
Output: trained parameters $\boldsymbol{\theta}^*$

1. Initialize network $\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}$ (Glorot)
2. Sample collocation sets $\mathscr{T}_r, \mathscr{T}_i\,[, \mathscr{T}_b]$
3. **for** $k = 1, \ldots, N_{\text{Adam}}$ **do**
4. forward pass at all sample points
5. input-derivatives via AD $\rightarrow$ assemble residual
6. $\mathscr{L} = w_r\mathscr{L}_r + w_i\mathscr{L}_i\,[+ w_b\mathscr{L}_b]\,[+ w_d\mathscr{L}_d]$
7. Adam step on $\nabla_{\boldsymbol{\theta}}\mathscr{L}$
8. optionally refine $\mathscr{T}_r$ (RAR), rebalance $\boldsymbol{\lambda}_\bullet$
9. **end for**
10. Refine with L-BFGS until convergence; **return** $\boldsymbol{\theta}^*$

Minimal code: a damped pendulum ODE-PINN

Solve $\ddot{\phi} + c\,\dot{\phi} + \frac{g}{\ell}\sin\phi = 0$ — the network maps $t \mapsto \hat{\phi}_{\boldsymbol{\theta}}(t)$:

```
import torch, torch.nn as nn
g_over_l, c = 9.81, 0.15
net = nn.Sequential(nn.Linear(1, 32), nn.Tanh(),
                    nn.Linear(32, 32), nn.Tanh(),
                    nn.Linear(32, 1))

def residual(t):                          # t: (N,1), requires_grad=True
    phi    = net(t)
    phi_t  = torch.autograd.grad(phi, t, torch.ones_like(phi),
                                 create_graph=True)[0]
    phi_tt = torch.autograd.grad(phi_t, t, torch.ones_like(phi_t),
                                 create_graph=True)[0]
    return phi_tt + c*phi_t + g_over_l*torch.sin(phi)
# loss = residual(t_col).pow(2).mean() + IC terms ...
```

05

One Recipe, Three Benchmarks

The same template, instantiated three times

Everything from Part I maps one-to-one onto the course benchmarks, in the order we take them:

	Heat diffusion	Surface vessel	Wind turbine & wake
Problem class	PDE, linear, 1-D	ODEs, nonlinear, coupled	ODE (drivetrain) + PDE (wake)
Unknown	$u(x,t)$, temperature	$\boldsymbol{\eta}(t), \boldsymbol{\nu}(t)$: pose, velocities	$\omega(t)$; deficit $d(x,y)$
Residual	$u_t - \alpha u_{xx}$	$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}\boldsymbol{\nu} + \mathbf{D}\boldsymbol{\nu} - \boldsymbol{\tau}$	$J\dot{\omega} - T_{\text{aero}} + K\omega^2$; $Ud_x - \nu_t d_{yy}$
IC / BC	$\sin \pi x$ / $u = 0$ at walls	IC only (hard)	IC (hard) / none
Data	60 sensors (inverse)	GNSS / IMU logs	SCADA rotor speed; LiDAR
Unknown $\boldsymbol{\lambda}$	diffusivity α	damping d_u, drag c	inertia J; eddy viscosity ν_t
Why here	exact reference	time-marching, identifiability	real-world, sparse data

What changes from column to column

Only the residual operator and the data source — everything else is *identical*. The columns go from simple to realistic.

06

Heat Diffusion (PDE)

The course benchmark, now without a grid

Recall the heat equation from the physics-based lecture, $\partial_t T = \alpha \nabla^2 T + \dot{q}/(\rho c_p)$. We use its simplest instance, where the exact solution is known:

$$u_t = \alpha\, u_{xx}, \quad x \in [0,1],\ t \in [0,1]; \qquad u(x,0) = \sin \pi x, \qquad u(0,t) = u(1,t) = 0; \qquad \alpha = 0.1.$$

Exact solution: $u(x,t) = e^{-\alpha \pi^2 t} \sin \pi x$. A PINN replaces the FTCS grid with a network $\hat{u}_{\boldsymbol{\theta}}(x,t)$. This is the *first member* ($c = e_1$) of the canonical rod family shared with Modules 02, 20, 21, 25 and 26.

Why start here

- Linear, smooth — the gentlest PDE-PINN
- Exact reference: score the *error*, not just the loss
- No CFL/von Neumann step-size limit

What stays hard (Part III)

- Balancing the loss weights
- Long time horizons
- Sharp fronts, multiple scales

Forward heat-PINN: the four residuals

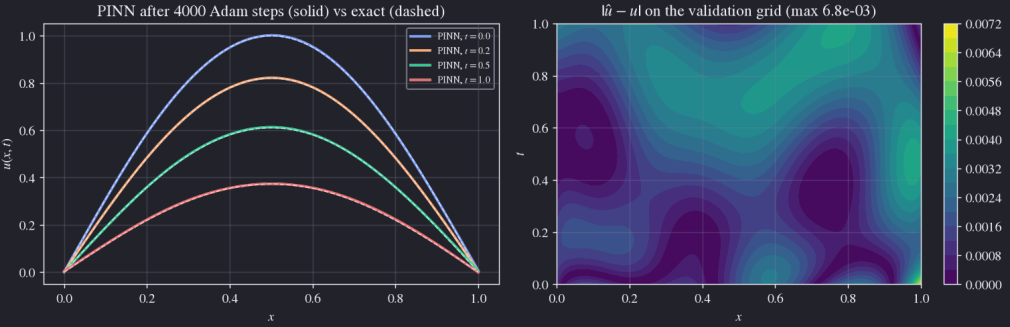
The anatomy slide, instantiated. Every residual is zero exactly when its condition holds:

PDE	$r(x,t) = \hat{u}_t - \alpha\,\hat{u}_{xx}$	2000 random points in $[0,1]^2$, redrawn every step
IC	$r_i(x) = \hat{u}(x,0) - \sin \pi x$	200 points on $t = 0$
BC	$r_b(t) = \hat{u}(0,t), \quad \hat{u}(1,t)$	200 points on each wall
Data	none	forward problem

$$\mathscr{L}(\boldsymbol{\theta}) = \overline{r^2} + 10\,\overline{r_i^2} + 10\left(\overline{\hat{u}(0,t)^2} + \overline{\hat{u}(1,t)^2}\right)$$

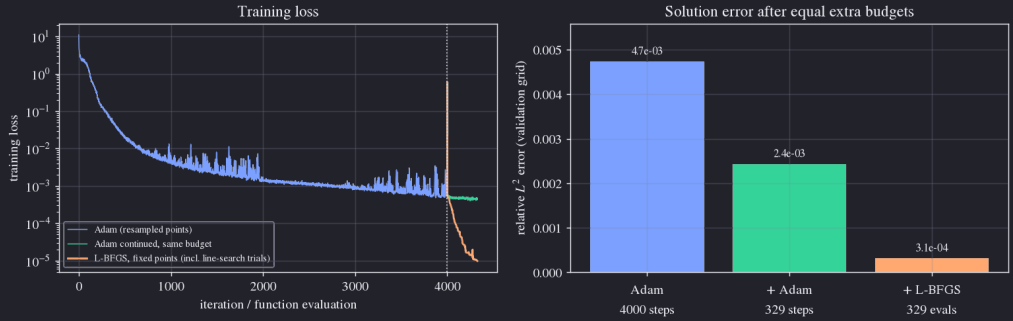
- Network $2 \to 32 \to 32 \to 32 \to 1$, tanh; $\hat{u}_t$, $\hat{u}_x$, $\hat{u}_{xx}$ by AD exactly as in Section 3.
- 4000 Adam steps (learning rate 2×10^{-3}, halved every 2000), then L-BFGS.

Forward heat-PINN: the result



Scored on a 101×101 grid the network never trained on: relative L^2 error 4.7×10^{-3} after Adam (4.7×10^{-3}–1.0×10^{-2} over five seeds). The final loss, 5.5×10^{-4}, says nothing about this number by itself (`pinn_heat_pde.ipynb`).

Polishing with L-BFGS



From the same Adam solution: 320 L-BFGS evaluations on a *fixed* point set cut the error $4.7 \times 10^{-3} \to 3.1 \times 10^{-4}$; 320 more steps of the same Adam run reach 2.1×10^{-3}. Over five seeds L-BFGS gained 9–23× and beat Adam every time. Note the left panel: Adam's loss barely moves while its error halves — loss and error are different measurements.

Inverse heat-PINN: recover the diffusivity

Now α is unknown. Only 60 noisy interior measurements are given — no IC, no BC:

$$\min_{\boldsymbol{\theta},\,\alpha} \overline{\left(\hat{u}_t - \alpha\,\hat{u}_{xx}\right)^2} + 100\;\overline{\left(\hat{u}(x_k,t_k) - u_k^{\text{data}}\right)^2}$$

Setup

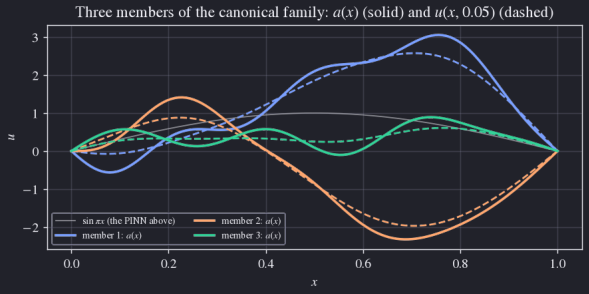
- $\log\alpha$ trained alongside $\boldsymbol{\theta}$, start at $\alpha = 0.5$
- Data: 60 samples, noise 0.01
- Physics fills the unmeasured space–time

Measured

$\alpha = 0.097$ (true 0.1): 3.3% error, and 1.4–3.5% over five seeds. Field error 1.5%, without ever being told the IC or BCs.

Hand-off: if *no* α reconciles physics and data, the equation is missing a term — the setting of CoSTA (Module 20).

One PINN, one profile: the question Module 25 answers



Three further members of the canonical family, $a(x) = \sum_k c_k \sin k\pi x$ (solid) and their exact $u(x, 0.05)$ (dashed). Nothing was trained for them.

- **What the network encodes:** one solution $u(x,t)$ for $u(x,0) = \sin \pi x$ — not the map from profile to solution.
- **What a new profile costs:** a new optimization from scratch (minutes), or at best an adaptation step from the current weights.
- **Two routes forward:** a *parameter-conditioned* PINN $\hat{u}_{\boldsymbol{\theta}}(x, t; \boldsymbol{c})$ trained over the family, or the *solution operator* of Module 25 (DeepONet) — with or without this residual in its loss (PI-DeepONet).

07

Autonomous Surface Vessels (ODEs)

Why model-based ML for autonomous vessels

Autonomy needs a dynamics model at every layer of the stack — and the model is never quite right.

Where the model is used

- Trajectory prediction & planning
- Model-predictive control (docking, path following)
- State estimation / observers
- Simulation for verification

Why it drifts from reality

- Loading condition changes draft & inertia
- Hull fouling alters damping
- Shallow water, bank effects
- Sea trials are expensive — data is scarce

The PINN fit: known model structure + sparse, noisy operational data → exactly the inverse-problem setting from Part I, now with *ordinary* differential equations.

Vessel dynamics: the 3-DOF maneuvering model

Surge–sway–yaw model in the standard (Fossen) form:

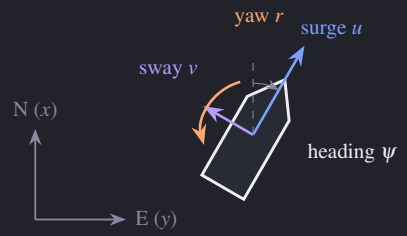
Kinematics + dynamics

$$\dot{\boldsymbol{\eta}} = \mathbf{R}(\psi)\,\boldsymbol{\nu}, \qquad \mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\,\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\,\boldsymbol{\nu} = \boldsymbol{\tau}$$

$\boldsymbol{\eta} = [x, y, \psi]^\top$: pose (NED); $\boldsymbol{\nu} = [u, v, r]^\top$: body velocities; $\boldsymbol{\tau}$: thruster forces/moment.

- $\mathbf{M} = \mathbf{M}_{RB} + \mathbf{M}_A$: rigid-body + added mass
- $\mathbf{C}(\boldsymbol{\nu})$: Coriolis–centripetal
- $\mathbf{D}(\boldsymbol{\nu})$: damping (linear + nonlinear)
- $\mathbf{R}(\psi)$: rotation, body $\rightarrow$ NED

Amber = uncertain hydrodynamic terms.



Forward ODE-PINN: trajectory prediction

Network maps time to the full state: $t \mapsto \big(\hat{\boldsymbol{\eta}}_{\boldsymbol{\theta}}(t), \hat{\boldsymbol{\nu}}_{\boldsymbol{\theta}}(t)\big)$, with the known input signal $\boldsymbol{\tau}(t)$.

Two residual vectors (six scalar equations)

$$\boldsymbol{r}_{\text{kin}} = \dot{\hat{\boldsymbol{\eta}}} - \mathbf{R}(\hat{\psi})\,\hat{\boldsymbol{\nu}}, \qquad \boldsymbol{r}_{\text{dyn}} = \mathbf{M}\,\dot{\hat{\boldsymbol{\nu}}} + \mathbf{C}(\hat{\boldsymbol{\nu}})\,\hat{\boldsymbol{\nu}} + \mathbf{D}(\hat{\boldsymbol{\nu}})\,\hat{\boldsymbol{\nu}} - \boldsymbol{\tau}$$

$$\mathscr{L} = w_r\big(\|\boldsymbol{r}_{\text{kin}}\|^2 + \|\boldsymbol{r}_{\text{dyn}}\|^2\big)_{\text{avg}} + w_i\big(\|\hat{\boldsymbol{\eta}}(0) - \boldsymbol{\eta}_0\|^2 + \|\hat{\boldsymbol{\nu}}(0) - \boldsymbol{\nu}_0\|^2\big)$$

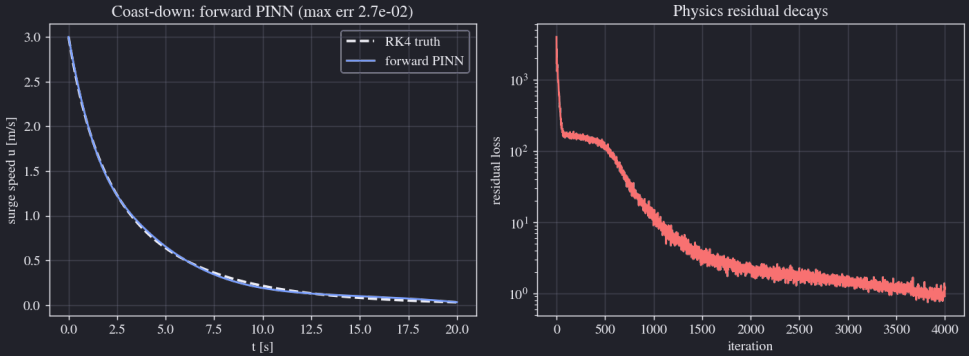
ODE-specific simplifications

- Collocation in *time only* — cheap
- No boundary loss at all
- Hard IC trick: $(\hat{\boldsymbol{\eta}}, \hat{\boldsymbol{\nu}})(t) = (\boldsymbol{\eta}_0, \boldsymbol{\nu}_0) + t \cdot \text{NN}(t)$ removes $\mathscr{L}_i$

ODE-specific cautions

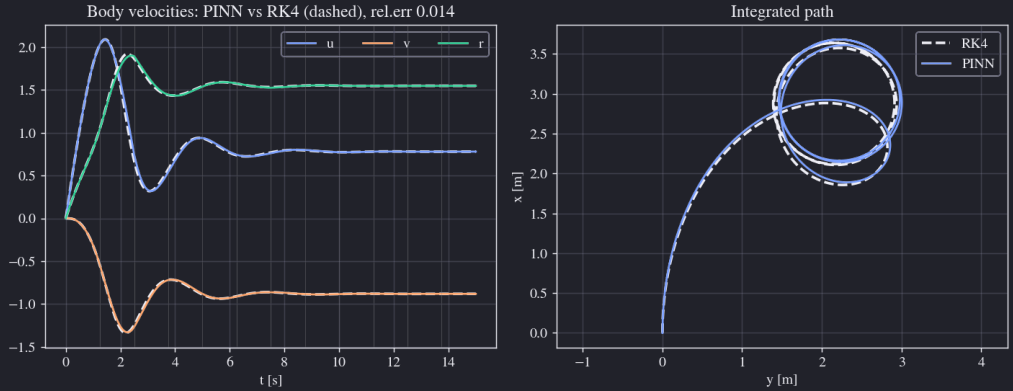
- Causality: late times fit before the transient
- Long maneuvers $\rightarrow$ time-marching windows
- Stiff thruster steps $\rightarrow$ denser sampling (RAR in 1-D)

Forward PINN, demonstrated: surge coast-down



The hard-IC ansatz $\hat{u}(t) = u_0 + t\,\mathrm{NN}(t)$ satisfies the initial condition exactly, so the loss is the physics residual alone. The PINN matches RK4 to a fraction of a percent (left) as the residual falls four orders of magnitude (right) (`pinn_vessel_ode.ipynb`).

Forward PINN scales to 3-DOF — by time-marching



The coupled surge–sway–yaw maneuver. A single global PINN stalls on the fast spin-up (a causality failure); solving short *time-marching* windows in sequence tracks the full transient to about one percent (left) and reproduces the path (right) (`pinn_vessel_ode.ipynb`).

Inverse ODE-PINN: hydrodynamic parameter identification

From logged maneuvers, recover the coefficients the model got wrong.

Setup

$$\min_{\boldsymbol{\theta},\boldsymbol{\lambda}} \mathscr{L}_r(\boldsymbol{\theta},\boldsymbol{\lambda}) + \mathscr{L}_d(\boldsymbol{\theta}), \qquad \boldsymbol{\lambda} = \{X_{\dot{u}}, Y_{\dot{v}}, N_{\dot{r}}, X_u, Y_v, N_r, X_{u|u|}, \dots\}$$

added-mass and damping derivatives entering $\mathbf{M}_A$ and $\mathbf{D}(\boldsymbol{\nu})$.

Data $\mathscr{L}_d$

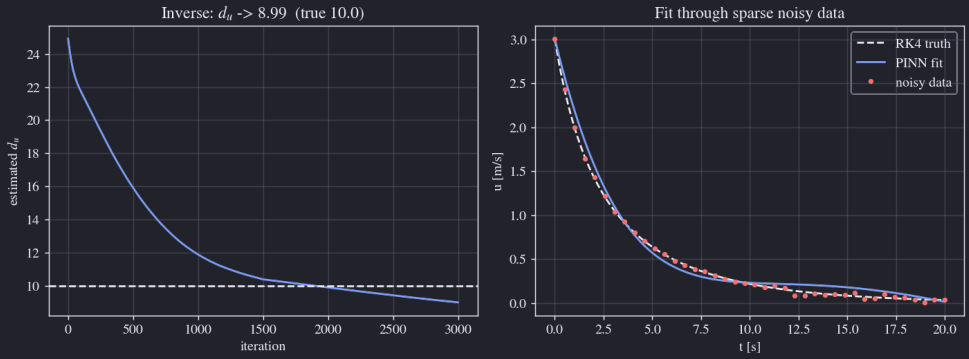
- GNSS: position, heading
- IMU: accelerations, yaw rate
- Thruster commands $\boldsymbol{\tau}(t)$ (known input)

Practical notes

- Excite all DOFs: zig-zags, turning circles; scale parameters
- Identifiability: some coefficients appear only in combination

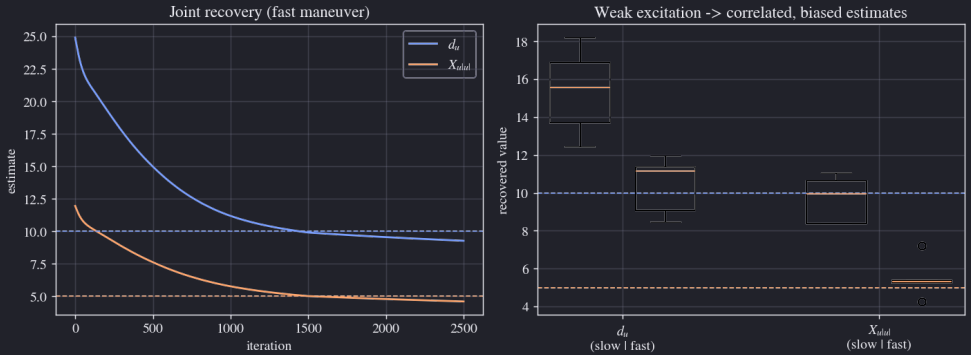
Direct analogue of the SCADA/LiDAR slide: same loss, different sensors, different $\boldsymbol{\lambda}$.

Inverse PINN, demonstrated: recover d_u



Making d_u a trainable parameter and adding a data-misfit on sparse noisy speed samples, the estimate moves from a wrong initial guess (25) to 9.0, within 10% of the true 10 (left), while the fit passes through the data (right) (`pinn_vessel_ode.ipynb`).

Identifiability: excitation decides

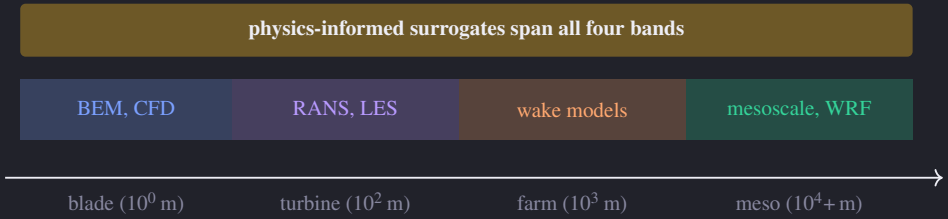


Recovering *both* damping coefficients: a slow maneuver leaves the linear and quadratic drag correlated, so estimates are wide and biased (left boxes of each pair); a fast maneuver excites the quadratic term and pins both down (right boxes) (`pinn_vessel_ode.ipynb`).

08

Wind Turbine and Wake (ODE + PDE)

Wind engineering: scales and governing models



- Different governing equations dominate at each scale — same PINN machinery, different N.
- Today: two representative problems — the turbine wake and the atmospheric boundary layer — plus the inverse / digital-twin workflow.

Wake modeling: the workhorse problem

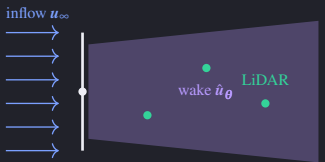
Predict the velocity deficit behind a turbine, conditioned on inflow.

Governing equations (steady RANS, actuator forcing)

$$(\boldsymbol{u}\cdot\nabla)\boldsymbol{u} = -\tfrac{1}{\rho}\nabla p + \nabla\cdot\left[(\nu+\nu_t)(\nabla\boldsymbol{u}+\nabla\boldsymbol{u}^{\top})\right] + \boldsymbol{f}_{\text{turb}}, \qquad \nabla\cdot\boldsymbol{u} = 0$$

What the PINN adds over Jensen / Gaussian models

- Continuous, *differentiable* wake field
- Handles inflow heterogeneity and yaw naturally
- Inverse mode: fit ν_t from sparse data



Differentiability matters: the wake surrogate can sit *inside* a layout or yaw optimizer — gradients flow through it.

Inverse problems from SCADA and LiDAR

Field data is sparse, noisy, and indirect — inverse PINNs turn this into an asset.

Setup

$$\min_{\boldsymbol{\theta},\boldsymbol{\lambda}} \; w_r \mathscr{L}_r(\boldsymbol{\theta},\boldsymbol{\lambda}) + w_d \mathscr{L}_d(\boldsymbol{\theta})$$

$\boldsymbol{\lambda}$: turbulence intensity, eddy-viscosity field, terrain roughness, thrust coefficient, …

Typical inputs

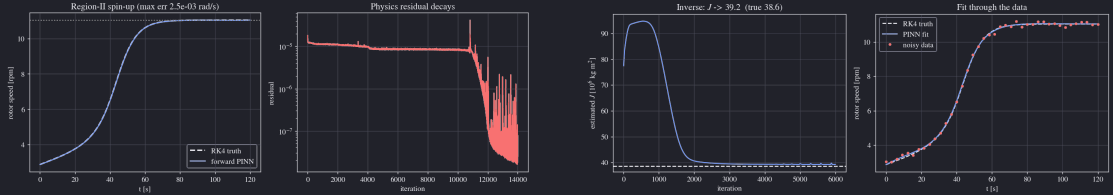
- SCADA: power, rotor speed, pitch, yaw ($\sim$1 Hz)
- Met-mast: 10-min statistics
- Nacelle / scanning LiDAR: line-of-sight velocities

Typical outputs

- Full 3-D flow field $\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}$
- Identified turbulence-model parameters
- Confidence intervals (ensembles, Part III)

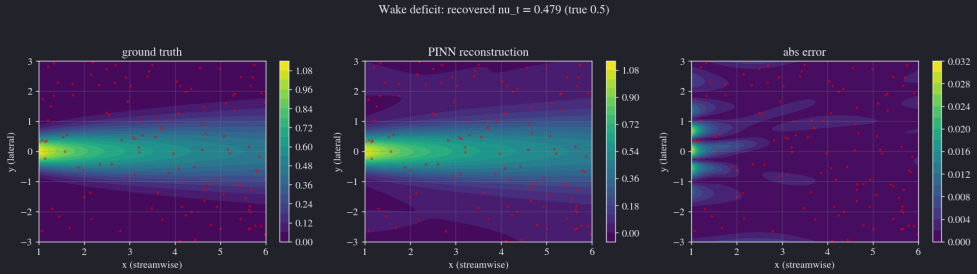
Wind result 1 — the turbine ODE-PINN

The drivetrain has its own ODE-PINN, exactly as the vessel did. Below rated, $J\dot{\omega} = T_{\text{aero}}(\omega, v) - K\omega^2$.



Left: the forward PINN matches the RK4 spin-up. *Right:* from noisy rotor-speed data it identifies the drivetrain inertia J (`pinn_wind_turbine.ipynb`).

Wind result 2 — the wake eddy viscosity



The PDE side: from sparse, noisy LiDAR-like deficit samples (red), the inverse PINN reconstructs the full wake field (1% relative error) and recovers the eddy viscosity: 4% error in this run, 4–19% over five seeds, always biased low — yielding a calibrated, *differentiable* wake for steering and layout (`pinn_wind_pde.ipynb`).

Atmospheric boundary layer: the inflow itself

Real farms sit in stratified, rotating, terrain-affected ABL flow.

Stratified ABL momentum equation

$$\frac{\partial \boldsymbol{u}}{\partial t} + (\boldsymbol{u}\cdot\nabla)\boldsymbol{u} = -\frac{1}{\rho_0}\nabla p + \nu\nabla^2\boldsymbol{u} \; \textcolor{blue}{-f_c \times \boldsymbol{u}} + g\,\textcolor{orange}{\frac{\theta-\theta_0}{\theta_0}}\,\boldsymbol{e}_z$$

Coriolis (blue) and buoyancy (orange); coupled to potential-temperature transport.

Stability regimes

- Neutral: log-law, well understood
- Stable (night): thin BL, low-level jets
- Convective (day): large eddies, intermittency

PINN role

- Physically consistent vertical profiles from sparse masts
- Inverse fitting of roughness z_0, friction velocity u_*
- Domain decomposition over terrain

09

Failure Modes & Remedies

Why your first PINN won't converge

PINNs look simple, but they exhibit characteristic pathologies that are not present in standard supervised learning.

1. Loss imbalance

$\mathscr{L}_r, \mathscr{L}_b, \mathscr{L}_d$ have wildly different scales & gradients $\rightarrow$ the network silently ignores the small ones.

2. Spectral bias

MLPs learn low frequencies first; sharp features (shocks, fast transients) train very slowly.

3. Causality violation

Time-dependent problems trained with all collocation points at once learn inconsistent "futures" before "pasts".

4. Stiffness

Stiff ODEs/PDEs $\rightarrow$ residual gradients explode; training stalls or diverges.

Loss balancing: adaptive weights

The relative magnitude of $\nabla_{\boldsymbol{\theta}} \mathscr{L}_r$ vs. $\nabla_{\boldsymbol{\theta}} \mathscr{L}_b$ decides which loss is *actually* minimized (Wang et al., 2021).

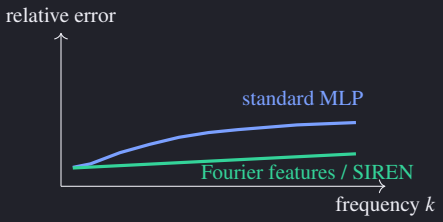
Gradient-norm balancing

$$\hat{\lambda}_i^{(k+1)} = \frac{\max \left| \nabla_{\boldsymbol{\theta}} \mathscr{L}_r^{(k)} \right|}{\overline{\left| \nabla_{\boldsymbol{\theta}} \mathscr{L}_i^{(k)} \right|}}, \qquad w_i^{(k+1)} = (1-\alpha)\, w_i^{(k)} + \alpha\, \hat{\lambda}_i^{(k+1)}$$

- Other schemes (SoftAdapt, self-adaptive per-point weights) follow the same idea: equalize the gradients, not the losses.
- Diagnose with separate loss curves and gradient-norm ratios over training.

Rule of thumb: if any loss component stays orders of magnitude above the others, your weights are wrong — automate them.

Spectral bias: why sharp features hurt



Remedies

- Fourier features: map inputs through $[\sin(\mathbf{B}\boldsymbol{x}), \cos(\mathbf{B}\boldsymbol{x})]$, random $\mathbf{B}$.
- SIREN: sinusoidal activations with careful initialization.
- Multi-scale branches at different frequency bands.

A smooth-activation MLP has a built-in preference for low frequencies. Work *with* it (non-dimensionalize, rescale) or override it (features above).

Causality and time-dependent problems

Vanilla PINNs treat all times symmetrically — but the dynamics have a direction.

Causal training (Wang et al., 2022)

Down-weight residuals at time t until earlier times are resolved:

$$w(t) = \exp\Big(-\varepsilon \sum_{t'<t} \mathscr{L}_r(t')\Big), \qquad \mathscr{L}_r^{\text{causal}} = \sum_t w(t)\,\mathscr{L}_r(t)$$

Alternative: time-marching

- Split $[0, T]$ into windows; one PINN per window, warm-started from the previous.
- Scales to long horizons.

Where we met it in Part II

- Vessel 3-DOF: one global PINN stalled on the fast spin-up; 12 time-marching windows gave 1.4% error.
- Wind: gusts and start-up transients have the same risk.

Failure-mode cheat sheet

Symptom	Likely cause	First remedy
Trivial constant solution	boundary/IC loss dominates	adaptive weights, hard constraints
Low loss, wrong solution	loss encodes another problem (ReLU $u_{xx} \equiv 0$; missing interface physics)	compare with a reference; check each residual
Smooth output, sharp truth	spectral bias	Fourier features, SIREN, RAR
Bad late-time predictions	causality violation	causal weights, time-marching
NaNs / diverging loss	stiffness or bad scaling	non-dimensionalize, clip, lower η
Plateaued residual	saddle / local minimum	switch to L-BFGS, restart
Good train, bad test	too few collocation pts	RAR, increase N_r

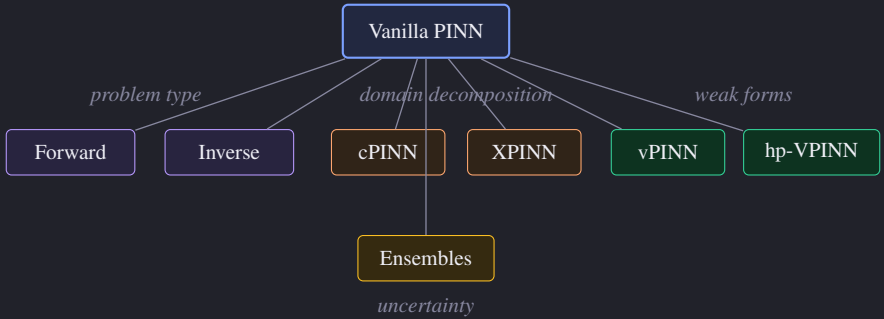
Diagnostic discipline

Always plot each loss component separately, the residual over the domain, and gradient-norm ratios. The aggregate loss alone hides everything that matters.

10

Algorithmic Variants

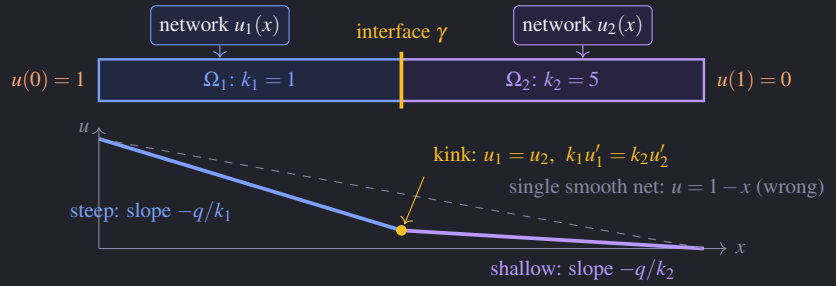
The PINN family tree



Every branch keeps the defining PINN ingredient — the residual in the loss — and changes one thing: where it is enforced, in what form, or with what confidence. Each variant answers a failure from the previous section; the next slides take one branch at a time.

cPINN: the problem — one rod, two materials

Steady conduction $\big(k(x)\,u'\big)' = 0$ through a rod whose conductivity jumps at $x = \gamma$:



$$\mathscr{L} = \overline{(u_1'')^2}_{\Omega_1} + \overline{(u_2'')^2}_{\Omega_2} + \big(u_1(0) - 1\big)^2 + u_2(1)^2 + \big(u_1(\gamma) - u_2(\gamma)\big)^2 + \big(k_1 u_1'(\gamma) - k_2 u_2'(\gamma)\big)^2$$

Inside each material the equation is just $u'' = 0$: k **appears only in the flux term**. A single network trained on $u'' = 0$ never sees the jump.

cPINN: one network per subdomain

Split Ω at an interface γ; give each subdomain its own network. The loss gains **interface terms**:

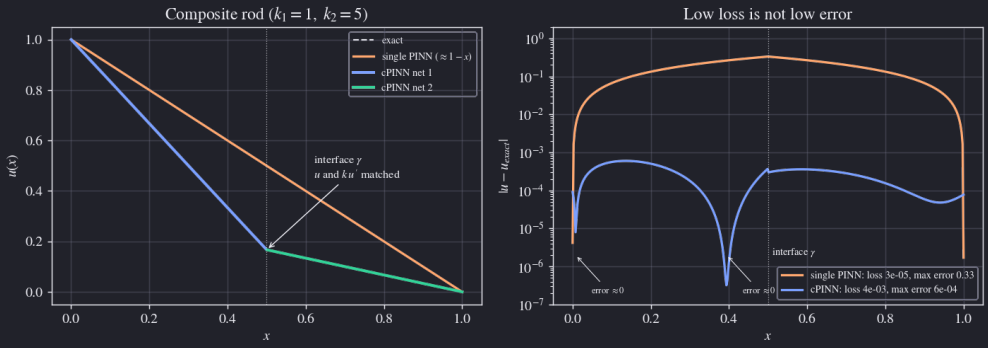
$$\mathcal{L} = \sum_{s=1}^{S} \mathcal{L}_r^{(s)} + \mathcal{L}_b + \underbrace{\big(u_1(\gamma) - u_2(\gamma)\big)^2}_{\text{state continuity}} + \underbrace{\big(k_1 u_1'(\gamma) - k_2 u_2'(\gamma)\big)^2}_{\text{flux (conservation)}}$$

- The flux term is the physics of the cut: whatever is conserved (heat flux, mass, momentum) must cross the interface intact — hence *conservative* PINN.
- Each network only represents a *simple* local solution — kinks, material jumps and multi-scale structure land on interfaces instead of inside one smooth net.
- Networks train in parallel; only the interface couples them.

When to reach for it

Piecewise materials, discontinuous coefficients, domains too large or too heterogeneous for one network.

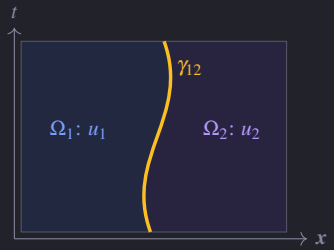
cPINN on the composite rod: low loss, wrong answer



The single network reaches a loss of 3×10^{-5} yet an error of 0.33: it found $u = 1 - x$, which satisfies $u'' = 0$ and both boundary values — a different problem. With the interface terms the error is below 10^{-3}. (The notches on the right are where the error touches zero; a log axis turns zero into a deep dip.) `xpinn_heat.ipynb`

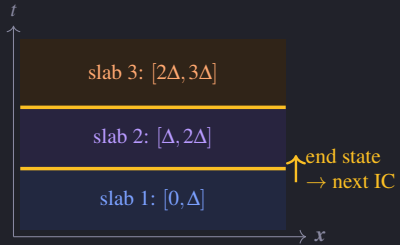
XPINN: the problem — cut space–time into pieces

Space decomposition



Any shape of cut, any PDE.

Time slabs



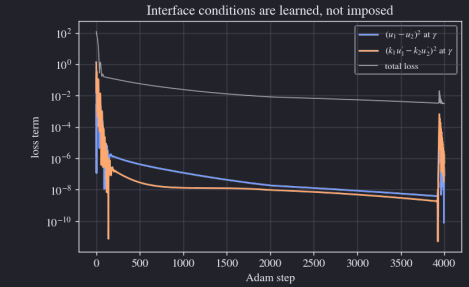
Solved in order: this is time-marching.

$$\mathscr{L} = \sum_s \mathscr{L}_r^{(s)} + \mathscr{L}_b + \mathscr{L}_i + \sum_{\gamma_{ij}} \left[\overline{\|\boldsymbol{u}_i - \boldsymbol{u}_j\|^2}_{\gamma_{ij}} + \overline{\|r_i - r_j\|^2}_{\gamma_{ij}} \right]$$

State continuity glues the solutions; residual continuity glues the dynamics, so it works for PDEs without a conserved flux. Met already: the vessel's 3-DOF solve used 12 slabs of 1.25 s (Part II).

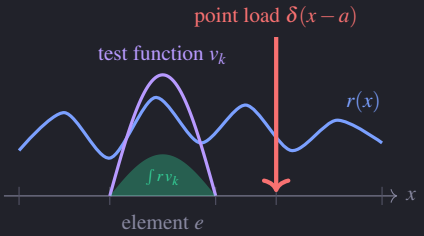
XPINN: the same glue, arbitrary decompositions

- Generalizes cPINN: any PDE (conservative or not), any space–*time* partition, plus a residual-continuity interface term $\big(r_1(\gamma) - r_2(\gamma)\big)^2$.
- **Time slabs are the important special case**: train window $[0, \Delta]$, then $[\Delta, 2\Delta]$ with the previous window's end state as initial data…
- …which is exactly the time-marching cure for the causality failure of the vessel's 3-DOF solve: errors are amplified over one *slab*, not over the whole horizon.



Interface losses during training: continuity and flux are *learned* down to 10^{-6}.

vPINN: the problem — a load with no pointwise value



Strong form (collocation): $r(x_j) = -u''(x_j) - f(x_j)$ needs f at points — but $\delta(x-a)$ has no value at $x = a$.

Weak form: average r against v_k, then integrate by parts:

$$\int_\Omega (-u'' - f)\, v_k \, dx = \int_\Omega u'\, v_k' \, dx - \int_\Omega f\, v_k \, dx = R_k$$

(boundary term vanishes: $v_k = 0$ at the element ends).

- u needs only *one* derivative
- the load enters as $\int \delta(x-a)\, v_k \, dx = v_k(a)$: a number
- loss $\mathscr{L} = \sum_k R_k^2$

vPINN / hp-VPINN: test the residual, do not collocate it

Multiply the residual by smooth test functions v_k and integrate by parts:

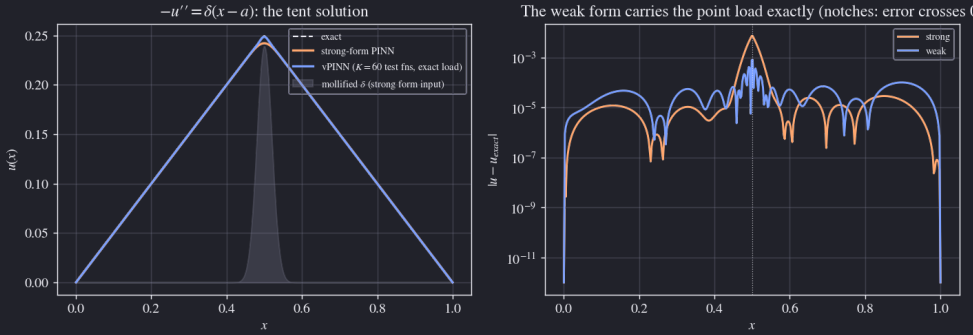
$$R_k = \int_\Omega u'_{\boldsymbol{\theta}}\, v'_k\, dx - \int_\Omega f\, v_k\, dx, \qquad \mathscr{L} = \sum_{k=1}^{K} R_k^2, \qquad v_k(x) = \sin(k\pi x).$$

- One derivative moves *off* the network onto v_k: cheaper autodiff, and $u_{\boldsymbol{\theta}}$ needs one order less smoothness.
- Singular or rough loads (f a point load $\delta(x-a)$, measured data, a distribution) are only ever *integrated*: $\int \delta(x-a)v_k = v_k(a)$ exactly, where the pointwise residual is meaningless.
- **hp-VPINN**: localize v_k per element (h-refinement) and raise the polynomial degree (p-refinement) — the FEM toolbox, imported wholesale.

Connection back

The same integration-by-parts trick is weak-form SINDy from Module 21 — there used to avoid differentiating noisy data.

vPINN on a point load



$-u'' = \delta(x-a)$: the strong form cannot even *state* the problem — it collocates a mollified Gaussian stand-in — while the weak form carries the load exactly as $v_k(a)$ and recovers the tent solution (`vpinn_heat.ipynb`).

Ensembles: uncertainty for the price of retraining

Retrain the *same* inverse PINN M times: fresh initialization, bootstrap-resampled data, fresh noise. Read the spread.

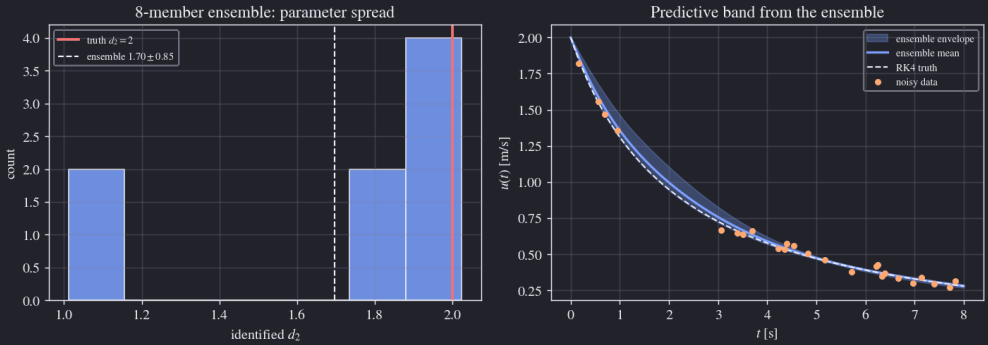
$$\{\hat{d}_2^{(m)}\}_{m=1}^{M} \;\Rightarrow\; \hat{d}_2 = \bar{d}_2 \pm 2\,\mathrm{std}\big(\hat{d}_2^{(m)}\big),$$

- Embarrassingly parallel; no new machinery beyond a for-loop.
- Captures **run-to-run variability** (initialization, noise, data resampling) — the dominant uncertainty in small-data inverse problems.
- What it does *not* capture: model-form error. A biased model is biased in every member; the band can be confidently wrong.

Seen already

The wind wake inverse problem (Part II) is exactly this: five seeds gave ν_t errors from 4% to 19%, all biased low — a spread one run would have hidden.

Ensemble inverse PINN: how sure are we about the drag?



Eight members identify the quadratic drag d_2 from 25 noisy coast-down samples: the histogram is the poor-man's posterior, the band the predictive envelope — truth inside both (`pinn_ensemble_vessel.ipynb`).

Choosing a variant

Failure you can name	Reach for	Notebook
Kinks, material jumps, big domains	cPINN / XPINN	`xpinn_heat`
Long-time dynamics, causality	XPINN time slabs	`pinn_vessel_ode`
Rough sources, low regularity	vPINN / hp-VPINN	`vpinn_heat`
"How sure are we?"	Ensembles	`pinn_ensemble_vessel`

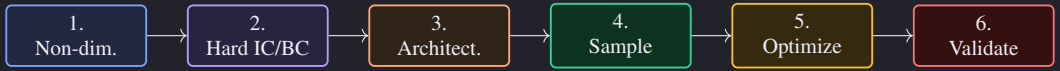
Rule of thumb

Start vanilla. Reach for a variant only when you can *name* the failure it fixes — every variant adds loss terms, weights, and tuning surface.

11

Practical Guidelines & Outlook

A workflow that actually works



1. Non-dimensionalize so all variables are $\mathcal{O}(1)$ — removes most stiffness for free.
2. Enforce ICs (and BCs) hard where possible — fewer loss terms, fewer weights.
3. Architecture to match the physics: depth ≥ 4, width ≥ 64; Fourier features for sharp content.
4. Latin hypercube by default; RAR for fronts and stiff transients.
5. Adam $\rightarrow$ L-BFGS, gradient-norm-balanced weights.
6. Validate against a reference solution or held-out maneuvers. Always.

Library landscape

Library	Backend	Strengths
DeepXDE	TF / PyTorch / JAX	most mature; many PDE & ODE examples
NVIDIA Modulus	PyTorch	production-grade; CFD-oriented tooling
PINA	PyTorch	modern API; operator-learning support
NeuralPDE.jl	Julia	symbolic; integrates with SciML / DifferentialEquations.jl
Custom (PyTorch/-JAX)	—	full control; recommended for ODE system ID

Recommendation

Wind / CFD-style PDE problems: start with **Modulus** or **DeepXDE**. Vessel ODE problems: a **custom PyTorch** loop (15–50 lines, as in Section 4) is usually clearer than any framework.

Open problems

Theory

- Convergence guarantees beyond linear problems
- Generalization in the small-data regime

Scalability

- 3-D turbulence at high Reynolds number
- Long-horizon ODE trajectories without windowing

Hybrid & certified models

- Coupling with FEM/FVM and classical integrators
- Guarantees suitable for safety-critical control

Foundation models

- Pre-trained operators; transfer across geometries, hulls, sites
- Physics-aware large-scale models

Certified guarantees are the gating question for marine autonomy.

Take-home messages

1. PINNs are one idea: put the residual in the loss. ODE or PDE, forward or inverse — the template does not change.
2. Part I was application-free *on purpose*: the same recipe solved heat, the vessel and wind — you can add a fourth benchmark yourself.
3. Failure modes are predictable and fixable: balance the losses, respect causality, scale your variables. Low loss is not low error — the ReLU heat PINN and the single-network rod both had tiny losses.
4. Wind: differentiable flow surrogates and parameter identification from SCADA/LiDAR.
5. ASVs: trajectory models, hydrodynamic coefficient ID, hybrid physics-plus-correction — inside the control loop.
6. Be honest about cost and accuracy. Validate. Always.
7. **Next**: one PINN solves one problem. Module 25 learns the *solution operator* for a whole family of inputs, from solver data or from this same residual; Module 26 does it on grids in Fourier space.



NTNU

Physics in the loss — one recipe, every benchmark

Adil Rasheed · NTNU · adil.rasheed@ntnu.no